UNIVERSITY INTEGRATION LAB

# Admissions Data Integration Pipeline

Moving applicant data from a REST API into PostgreSQL — without duplicates, stale overwrites or silent failures.

Alison Gorman · Independent portfolio project · October 2026

THE PROBLEM

# Moving data between systems fails in three predictable ways

### Duplicates

Re-running a load inserts the same applicant twice.

### Stale overwrites

An older extract replaces newer data that is already in the database.

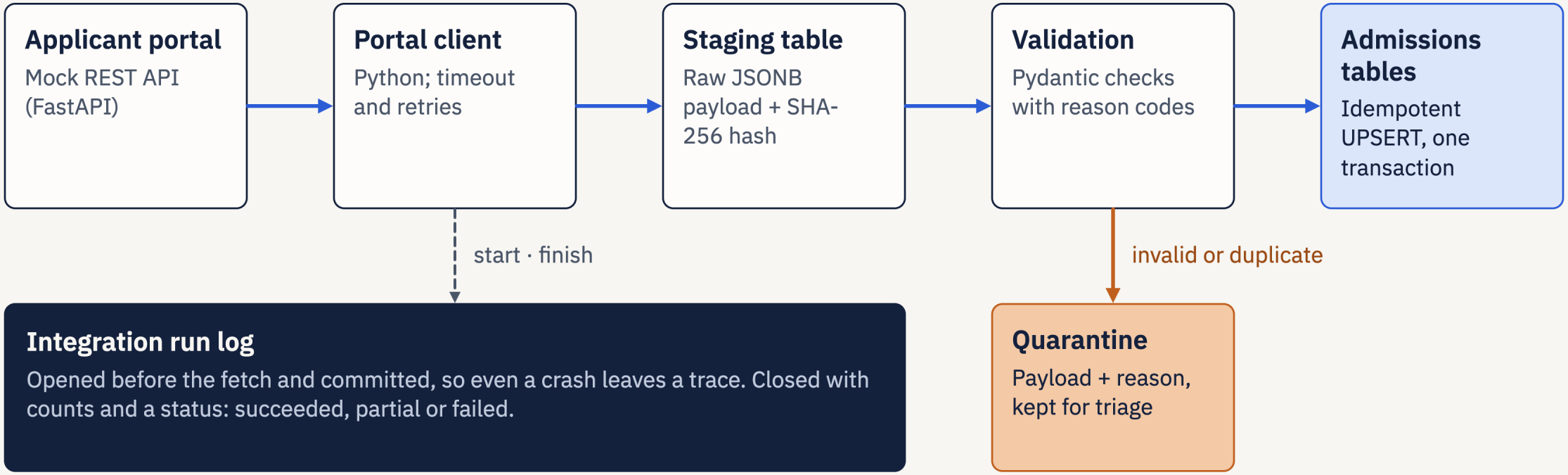
### Silent failures

A bad record or an API timeout drops data with no trace.

This pipeline **prevents** the first two and makes the third **visible and recoverable**.

ARCHITECTURE

# One pipeline, five stages




Solid blue = data flow · Orange = rejected records · Dashed = run logging

STEP 1 · FETCH

# API calls are retried, not assumed to succeed

01
**Request**
GET /applications with a 5 s timeout.

→

02
**Transient error**
A connection error or a 5xx response.

→

03
**Back off & retry**
Up to 3 retries, waiting 0.5 s, 1 s, then 2 s.

→

04
**Success**
Records are staged exactly as received.

⚠ **Still down?** The run is marked failed with the error message and nothing else is written. A 4xx fails fast: it means our request is wrong.

STEP 2 · STAGE & VALIDATE

# Bad records are quarantined, not dropped

## Passes validation

Promoted from staging into the admissions tables.

## Fails → quarantine, with a reason code

| Reason code | Example |
|---|---|
| missing_required_field | No last name |
| invalid_format | A malformed email |
| unknown_program | Program code not on file |
| invalid_status | Status outside the allowed set |
| duplicate_in_batch | Older copy of a repeated record |

**One bad record never fails the batch.** Its payload and reason stay in quarantine for triage and replay.

STEP 3 · PROMOTE

# Idempotent UPSERTs stop duplicates and stale overwrites

```
INSERT INTO admissions.application AS t
  (portal_application_id, …, source_updated_at)
SELECT … FROM admissions.program AS p …
ON CONFLICT (portal_application_id) DO UPDATE
SET status = EXCLUDED.status, …,
    source_updated_at = EXCLUDED.source_updated_at
WHERE EXCLUDED.source_updated_at > t.source_updated_at
  AND (t.status, …) IS DISTINCT FROM
      (EXCLUDED.status, …);
```

| Incoming record | Result |
|---|---|
| New portal ID | Insert |
| Known ID, newer, changed | Update |
| Known ID, newer, same data | Skip (no-op) |
| Known ID, older | Skip (stale) |

The batch runs as **one transaction**. If the database rejects it, the job retries **row by row with SAVEPOINTs** and quarantines only the offending record.

STEP 4 · TROUBLESHOOT

# Every run leaves a trail you can query

## integration_run

- Status: running, succeeded, partial or failed
- Records fetched, inserted, updated, quarantined
- Error message when a run fails
- Every log line tagged [run N]

## A runbook: symptom → query → fix

Portal down: why did the last run fail?

Which records were quarantined, and why?

Are two applicants really the same person?

Which rows are stuck in staging?

BUILD

# Tech stack and the job each piece does

### Python 3.12

The lab-sync job: fetch, stage, validate, promote

### PostgreSQL 16

Staging, quarantine, run log and admissions tables

### SQL

UPSERTs, the promote transaction and troubleshooting queries

### REST APIs

GET /applications over HTTP with httpx

### FastAPI

Mock applicant portal with fictional Faker data

### Docker

Compose runs PostgreSQL 16 and applies the schema

### pytest

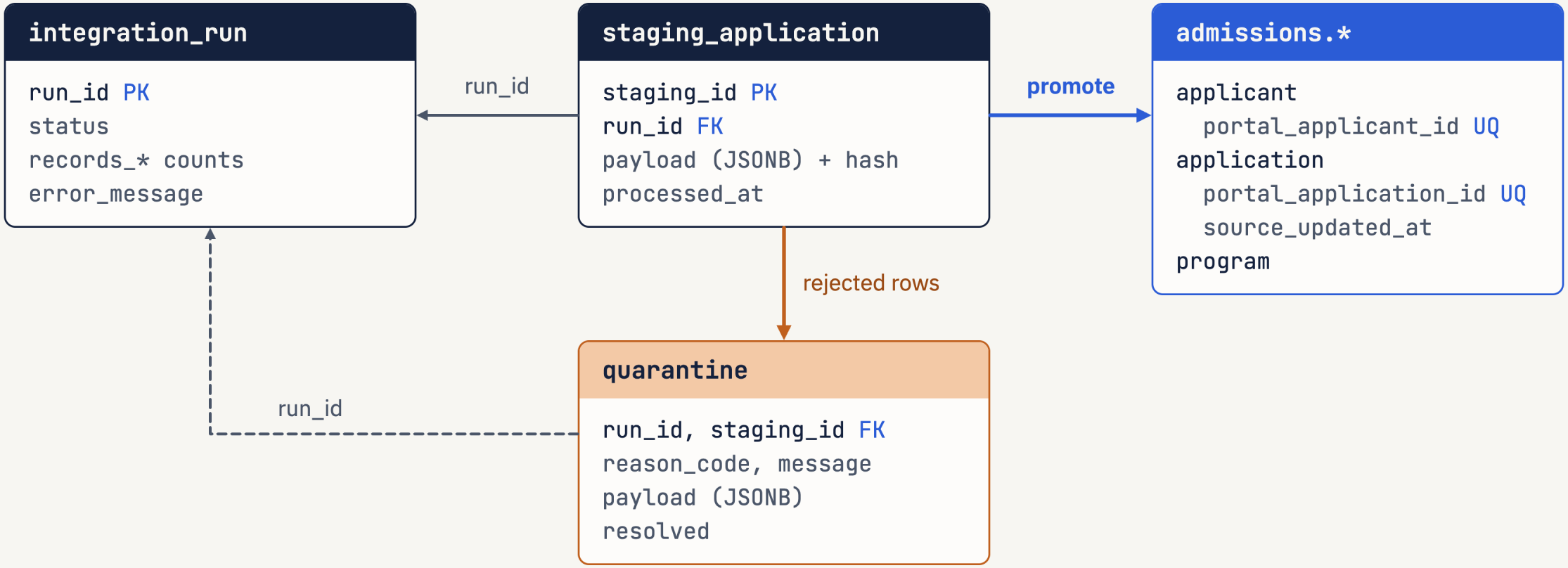
19 tests: duplicates, bad data, API and database failures

### GitHub Actions

ruff and pytest against PostgreSQL 16 on every push

DATA MODEL

# Two schemas: integration and admissions




Simplified · Full schema with a written decision per design question: db/ddl/001_schema.sql

TAKEAWAYS

# Built the way production integrations need to run

## Reliable

Idempotent UPSERTs keyed on source IDs

Timestamp guards against stale data

## Recoverable

Retries for transient API errors

Quarantine and replay for bad records

## Supportable

A run log for every execution

A runbook, troubleshooting SQL and 19 tests in CI

Alison Gorman · University Integration Lab